推理模式：Agent如何思考

- 来源: https://class.imooc.com/lesson/5290#mid=105152
- 阶段: 阶段六: 【智能体】从单体到多体
- 周: 第12周 Multi-Agent：多智能体协作
- 模块: Agent的核心架构剖析
- 课节: 图文 1-4 推理模式：Agent如何思考

1-4 推理模式：Agent如何思考

推理模式：Agent如何思考

> [!TIP] 阅读指南 前面几节，我们已经了解了Agent的整体架构、感知能力、规划能力和记忆系统。但有一个核心问题始终没有正面回答: Agent是如何"思考"的? 当Agent面对一个问题时，它在大脑(LLM)中经历了什么过程?为什么有时能给出完美答案，有时却会犯低级错误?

推理与感知-规划-执行的关系

推理在Agent的"感知-规划-执行"循环中处于什么位置?

答案是:推理贯穿整个循环的每一步。

推理贯穿整个循环的每一步

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8

- 9

具体来说:

具体来说

- 感知阶段的推理: “用户说’帮我订票’ → 推理出这是预订需求 → 需要询问目的地和时间”
- 规划阶段的推理: “要订票 → 需要先查天气 → 再选交通工具 → 最后预订” (这是CoT推理)
- 执行阶段的推理: “调用天气API失败 → 推理出可能网络问题 → 决定重试还是换备用API” (这是Self-Reflection)

所以，推理不是独立的一步，而是Agent在每个决策点都要用到的核心能力。

推理不是独立的一步，而是Agent在每个决策点都要用到的核心能力

本节我们聚焦的是:Agent在这些决策点使用的推理模式(CoT、ToT、ReAct等)有什么特点，如何选择和组合使用。

Agent推理的核心挑战

在第1章第10节，我们学习了Chain-of-Thought(思维链)的基础原理——通过"让我们一步步思考"引导LLM展示推理过程，能让准确率从17%提升到78%。

但Agent面临的推理场景远比单次问答复杂:

```
单次问答(第1章场景):
用户提问 → LLM推理 → 给出答案 → 结束

Agent场景(本章关注点):
用户目标 → 多轮推理决策 → 调用工具 → 观察结果
         ↑                          |
         └──────── 循环往复 ─────────┘
```

Agent推理的三大特殊性:

Agent推理的三大特殊性

- 动态性:每一轮的推理都基于上一轮的观察结果

动态性

- 工具依赖:需要决策何时调用哪个工具

工具依赖

- 多轮性:可能需要5-10轮推理才能完成任务

多轮性

这就需要更强大的推理模式。

CoT在Agent中的特殊应用

第1章我们学过，CoT有两种形式:Zero-Shot(“让我们一步步思考”)和Few-Shot(给推理示例)。但在讲Agent中的CoT应用之前，我们必须先扔掉一个常见的误解。

CoT的本质:不是"思考",而是"生成文本"

很多人看到CoT的例子后，会认为:LLM真的在“思考”。但事实是:LLM只是在生成一段看起来像推理过程的文本。

当你给LLM这样的Prompt:

```
问题:小明有3个苹果,又买2个,他现在有几个苹果?

让我们一步步思考:
```

LLM会生成这样的文本:

```
小明最开始有3个苹果
他又买2个
所以总数是 3 + 2 = 5
答案:5个苹果
```

看起来LLM像是一个人一样在“思考”， 但其实不是。LLM只是在生成文本，他根本不知道这是连续的思维过程。具体来说:

- LLM并没有真的在"计算" 3+2
- LLM只是在训练数据中见过大量“问题→推理步骤→答案”的样本
- 它学会了这种文本模式,所以能生成类似的输出

文本模式

回顾：CoT如何让LLM生成"推理步骤"

Zero-Shot CoT:只需加一句引导词

Zero-Shot CoT

```
# 不用CoT
prompt = "小明有3个苹果,又买2个,他现在有几个?"
response = llm.invoke(prompt)
# 输出:“5个”(直接给答案,但可能出错)

# 使用CoT
prompt = """
小明有3个苹果,又买2个,他现在有几个?

让我们一步步思考:
"""
response = llm.invoke(prompt)
# 输出:“步骤1:...步骤2:...答案:5个”(生成了推理步骤)
```

为什么有效

- 训练数据中有大量“让我们一步步思考”后面跟着详细推理的样本
- LLM学会了:看到这句话 → 应该生成步骤式的回答

Few-Shot CoT:给几个示例

Few-Shot CoT

```
prompt = """
示例1:
问题:小红有5个橘子,吃掉2个,还剩几个?
推理:
- 最开始:5个
- 吃掉2个:减去2个
- 计算:5 - 2 = 3
答案:3个

示例2:
问题:小明有3支笔,又买2支,现在有几支?
推理:
- 最开始:3支
- 又买2支:增加2支
- 计算:3 + 2 = 5
答案:5支

现在轮到你:
问题:小明有3个苹果,又买2个,他现在有几个?
"""
response = llm.invoke(prompt)
# LLM会模仿示例的格式,生成类似的推理步骤
```

CoT的核心原理在于生成文本模式，而非真正的逻辑推理。

CoT在Agent中的应用:任务规划

现在我们来看，CoT在Agent系统中是如何应用的。Agent中的CoT主要用于任务规划阶段,而不是执行阶段。

用户问：

> 帮我查一下明天杭州和上海的天气，推荐一个适合旅游的城市。

Agent 在规划阶段把用户问题包装成 CoT 风格的 Prompt，发给 LLM：

```
用户问题：帮我查一下明天杭州和上海的天气，推荐一个适合旅游的城市。

请制定一个任务执行计划，一步步分析需要做什么：
```

LLM 收到后，生成如下计划文本：

> 步骤1：理解需求 用户问了两个城市的天气 需要比较后给出推荐 步骤2：分解子任务 子任务1：查询杭州天气 子任务2：查询上海天气 子任务3：对比分析 子任务4：给出推荐 步骤3：选择工具 需要用 get_weather 工具查询天气 查询两次（杭州和上海）

注意：

注意

- 这里的“步骤1、步骤2、步骤3”是 LLM 生成的文本

生成的文本

- 这是一个计划，不是真实执行
- 真实执行是后面的事（会用到 ReAct 模式，后面章节会讲）

Tree-of-Thought:Agent的多路径探索

在Agent系统中，经常遇到需要尝试多个工具或多种方案的场景。这时单路径的CoT就不够用了，需要Tree-of-Thought(思维树)来探索最优解。

ToT的Agent应用场景

场景1:多工具选择

```
用户:帮我分析一下公司这个月的销售数据,找出top5产品,
    并生成一份可视化报告发给老板

[第1层:选择数据源工具]
  选项1:read_excel读取本地文件
  选项2:query_database查询数据库
  选项3:call_api调用销售系统API

  评估:用户说"这个月",需要实时数据 → 选API

[第2层:选择分析工具]
  选项A:直接用pandas计算
  选项B:调用数据分析服务
  选项C:用SQL聚合

  评估:数据量不大,本地处理即可 → 选pandas

[第3层:选择可视化工具]
  格式1:generate_chart生成静态图
  格式2:create_dashboard生成交互式看板
  格式3:直接返回表格

  评估:给老板看,需要专业呈现 → 选交互式看板

[第4层:选择发送方式]
  方式1:send_email发邮件
  方式2:upload_to_drive上传到共享盘
  方式3:post_to_slack发到工作群

  评估:用户说"发给老板",正式场合 → 选邮件

最终方案:API取数 → pandas分析 → 交互看板 → 邮件发送
```

场景2:异常处理策略

```
工具调用失败时,探索多种补救方案:

[第1轮:天气API调用失败]
  方案1:重试3次
  方案2:切换备用API
  方案3:使用缓存数据

  评估:备用API可能也失败 → 选重试

[第2轮:重试仍失败]
  方案A:返回错误给用户
  方案B:用历史天气数据估算
  方案C:改用网页爬取

  评估:历史数据不准确 → 选网页爬取

最终路径:重试 → 网页爬取 → 成功获取数据
```

CoT与ToT的实现原理区别

CoT的实现:Prompt层面的引导

CoT本质上非常简单，只是在Prompt中加入引导词:

在Prompt中加入引导词

```
Zero-Shot CoT:
用户问题 + "让我们一步步思考"
  ↓
单次LLM调用
  ↓
LLM自动生成:步骤1→步骤2→步骤3→答案

Few-Shot CoT:
示例1(问题→推理→答案) + 示例2 + 用户问题
  ↓
单次LLM调用
  ↓
LLM模仿示例格式输出推理过程
```

关键特点:

关键特点

- 单路径:LLM只生成一条推理链
- 单次调用:整个过程只需1次API调用

单次调用

- 无需算法:纯Prompt技巧，任何支持对话的LLM都能用

无需算法

ToT的实现:需要搜索算法控制

ToT复杂得多，需要外部程序控制的树搜索算法，所以我们不推荐手工实现ToT。这里我们只简单介绍下ToT的关键核心技术:

```
核心流程:
1. 生成层:让LLM为同一问题生成多个不同的初步想法
2. 评估层:让LLM或设计评估函数给每个想法打分
3. 选择层:用搜索算法选择最优的几个分支继续扩展
4. 递归层:对选中的想法继续步骤1-3,构建多层决策树
5. 最终选择:从所有路径中选出得分最高的完整方案
```

需要的关键算法:

需要的关键算法

- 广度优先搜索(BFS):每层探索所有可能再筛选

广度优先搜索(BFS)

- 深度优先搜索(DFS):沿一条路径探索到底再回溯

深度优先搜索(DFS)

- 蒙特卡洛树搜索(MCTS):结合随机采样和价值评估

蒙特卡洛树搜索(MCTS)

- 剪枝算法:避免组合爆炸，只保留最有希望的k个分支

剪枝算法

- 树状结构:每层生成3-5个分支，探索不同可能性

树状结构

- 多次调用:每个节点都要调用LLM(可能10-30次)

多次调用

- 需要评估:每个想法都要打分，判断优劣

需要评估

- 算法复杂:需要实现完整的树搜索框架

算法复杂

如果你需要在Agent系统中使用ToT,推荐LangGraph。官方文档提供完整的ToT教程和示例代码。

最佳实践:

最佳实践

- 生产环境优先选择LangGraph实现:官方支持、生态成熟

生产环境优先选择LangGraph实现

- 成本考量:ToT的多次LLM调用成本较高，评估是否真的需要

成本考量

- 适用场景:只在关键决策点(如多工具选择、复杂规划)使用ToT

适用场景

- 大多数情况:CoT配合Self-Reflection（下节讲解）就足够了

大多数情况

Self-Reflection:Agent的自我修正

Agent需要反思的原因

Agent的工具调用和多轮交互尤其容易出错:

```
用户:帮我订明天去杭州的高铁票。

Agent(没有反思):
调用:book_train("明天", "杭州")
结果:失败 - 缺少出发城市参数

Agent(有反思):
反思:要订票需要出发城市,用户没提供,我应该先询问。
行动:询问用户出发城市
```

Agent反思的三个阶段

在调用工具前，Agent先反思:

```
用户:帮我查一下明天杭州的天气,如果不下雨就订票。

Agent:
我要调用get_weather工具,参数是:
- 城市:杭州
- 日期:明天

反思:等等,"明天"不是标准日期格式,API可能无法识别。

修正:先转换为"2024-12-10",再调用工具。
```

工具调用后，检查结果是否合理:

```
Agent调用:get_weather("杭州", "2024-12-10")
返回:{"weather": "晴天", "temperature": "5-15度"}

反思:天气不错,可以订票。但用户没说从哪里出发,
不能直接调用book_train。

修正:先询问用户出发城市。
```

在给出最终答案前，检查逻辑是否合理:

```
用户补充:从上海出发。

Agent已订票:上海 → 杭州,12月10日,G7533次

Agent初步答案:已为您订好明天的票,上海到杭州。

反思:等等,我还没告诉用户天气情况,
这是用户决策的重要依据。

修正:明天杭州晴天5-15度,已为您订好上海到杭州的G7533次列车。
```

成本权衡

Agent中的反思需要谨慎使用:

- 成本增加:每次反思都是额外的LLM调用

成本增加

- 延迟增加:用户等待时间更长

延迟增加

- 过度反思:可能陷入"纠结循环"

过度反思

Agent最佳实践:

Agent最佳实践

- 关键步骤才反思(工具调用、最终答案)
- 设置反思上限(最多3次) 异常触发反思(正常情况不反思)

下一节预告

理解了Agent的推理模式后，你可能会迫不及待地想动手实现一个自己的Agent。但这时会面临一个关键选择：是从零手写代码，还是使用现成框架？用单体架构简单直接，还是模块化设计更易扩展？接下来，我们将对比Agent的各种实现范式，帮你根据实际场景做出最适合的技术选型。

■ 学点英语

中文	English	音标	说明
思维链	Chain-of-Thought (CoT)	/tʃeɪn əv θɔːt/	通过一步步推理展示LLM决策过程的Prompt技巧
思维树	Tree-of-Thought (ToT)	/triː əv θɔːt/	探索多条推理路径并选择最优解的搜索框架
自我反思	Self-Reflection	/self rɪˈflekʃn/	Agent在关键节点自行检查并修正决策的机制
广度优先搜索	BFS (Breadth-First Search)	/bredθ fɜːrst sɜːrtʃ/	ToT中每层探索所有可能再筛选的搜索算法
深度优先搜索	DFS (Depth-First Search)	/depθ fɜːrst sɜːrtʃ/	ToT中沿一条路径探索到底再回溯的搜索算法
蒙特卡洛树搜索	MCTS (Monte Carlo Tree Search)	/em siː tiː es/	结合随机采样和价值评估的树搜索算法